DEEP LEARNING | SEMESTER PROJECT REPORT

FloraScan AI

Smart Phytopathology System

Dual-engine tomato leaf disease classification with a custom CNN and transfer-learned ResNet-18

5 classes | 5,000 images | 96.53% test accuracy

SUBMITTED BY

Asad Ali Sajjad

Roll No. SU92-BSCSM-F23-167

BSCS | Section 7A | 7th Semester

SUBMITTED TO

Dr. Munir Ahmad

Course: Deep Learning

Final Technical Report

PyTorch | Torchvision | Streamlit

OVERVIEW

Contents and Abstract

1	Introduction and Problem Statement
2	Dataset Curation and Preprocessing
3	Model Architectures
4	Training Setup
5	Results and Evaluation
6	Analysis and Discussion
7	Application and System Design
8	Limitations and Future Work
9	Conclusion
	References

Abstract

Plant diseases are a major driver of crop loss, and early, accurate identification is the first step toward containing them. FloraScan AI is an end-to-end deep learning system that classifies tomato leaf health into five states: Healthy, Bacterial Spot, Early Blight, Late Blight, and a synthetic *Not Leaf* class that lets the system reject images that are not leaves at all. Two models were trained on a balanced 5,000-image subset of the PlantVillage dataset: a custom three-block convolutional network trained from scratch, and an ImageNet-pretrained ResNet-18 whose backbone was frozen. On a held-out test set of 750 images the custom CNN reached **96.53%** accuracy (724 correct) and ResNet-18 reached **94.13%** (706 correct). Both models are served through a Streamlit application that offers single-engine inference, a dual-engine agreement check, a 65% confidence safety threshold, treatment guidance, and downloadable reports.

96.53%	94.13%	5,000	5
Custom CNN test accuracy	ResNet-18 test accuracy	Curated images	Output classes

BACKGROUND

1. Introduction and Problem Statement

Tomato is one of the most widely grown vegetable crops, and it is also one of the most disease-prone. Bacterial and fungal-like pathogens spread quickly through a field, and by the time symptoms are obvious the damage is often hard to reverse. Industry bodies such as the FAO estimate that pests and diseases cost global agriculture a substantial share of potential yield each year, with figures of 20 to 40 percent commonly cited. Smallholder farmers feel this loss most, because expert inspection is slow, scarce, and sometimes subjective.

Computer vision offers a practical alternative. A photograph of a single leaf contains enough colour, texture, and lesion-shape information for a trained network to separate common diseases. The goal of this project is to build and evaluate that capability from start to finish, and to wrap it in an interface a non-expert can use.

Objectives

- Curate a clean, balanced dataset from PlantVillage covering three tomato diseases and healthy leaves.
- Add an out-of-distribution *Not Leaf* class so the system can refuse obviously invalid inputs.
- Train and compare a custom CNN built from scratch against a transfer-learning ResNet-18 baseline.
- Deploy both models in a web application with a confidence guardrail and actionable guidance.



Figure 1. Project workflow from raw data to deployed application.

DATA

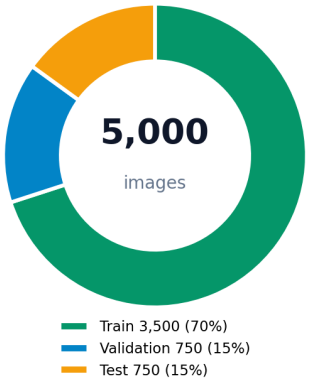
2. Dataset Curation and Preprocessing

The source is the PlantVillage corpus: 20,638 images across 15 crop-disease categories (pepper, potato, and tomato). Class sizes in the raw data are heavily uneven; for example Tomato Yellow Leaf Curl Virus has 3,208 images while Potato healthy has only 152. To avoid a model that simply favours the largest class, four tomato categories were selected and capped at **1,000 randomly sampled images each**. A fifth class, *Not Leaf*, was generated programmatically.

Class	Images	Source	What it represents
Tomato Healthy	1,000	PlantVillage	Even green tissue, intact margins, no lesions.
Tomato Bacterial Spot	1,000	PlantVillage	Small dark, water-soaked spots, often with yellow halos (*Xanthomonas*).
Tomato Early Blight	1,000	PlantVillage	Target-like concentric rings on older leaves (*Alternaria solani*).
Tomato Late Blight	1,000	PlantVillage	Large grey-green, water-soaked lesions that spread rapidly (*Phytophthora infestans*).
Not Leaf	1,000	Synthetic	Solid colours, random noise, checkerboards, and geometric shapes (five patterns, 200 each).

Splitting and augmentation

The 5,000 images were divided 70 / 15 / 15 into training (3,500), validation (750), and test (750) sets using a seeded random split (seed 42) for reproducibility. Training images receive a random horizontal flip and a random rotation of up to 15 degrees. Validation and test images are only resized and normalised, so evaluation stays deterministic. All images are resized to 224 x 224 and normalised with ImageNet statistics (mean 0.485, 0.456, 0.406; std 0.229, 0.224, 0.225).



Why a synthetic class?

A classifier trained only on leaf classes must always pick a leaf label, even for a photo of a hand or a wall. The synthetic *Not Leaf* class teaches the network that "none of the above" is a valid answer. Together with the 65% confidence threshold in the app, this reduces the chance of a confident but meaningless diagnosis.

Note that these synthetic images are simple, so the class mainly guards against clearly non-botanical inputs rather than real-world clutter such as soil or hands.

MODELS

3. Model Architectures

Two networks were trained under identical conditions so that the comparison reflects the modelling approach rather than the training recipe.

A. Custom three-block CNN (CustomPlantCNN)

Three convolution blocks progressively halve the spatial resolution (224, 112, 56, 28) while increasing channel depth (32, 64, 128). Each block applies a 3 x 3 convolution, batch normalisation, ReLU, and 2 x 2 max pooling. The 128 x 28 x 28 feature map is flattened into a 100,352-dimensional vector and passed through a 512-unit fully connected layer with batch normalisation, ReLU and dropout (0.25), followed by the 5-way output layer.

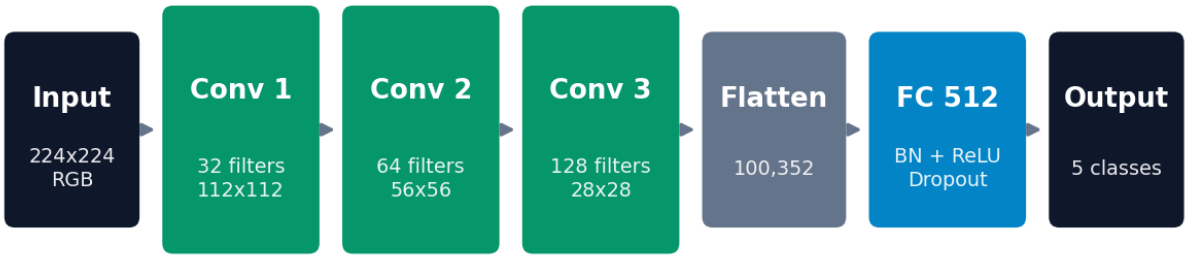


Figure 2. CustomPlantCNN: all layers are trained from scratch.

B. Transfer learning with ResNet-18

ResNet-18 is loaded with ImageNet weights. Every pretrained parameter is frozen and the final fully connected layer is replaced with a new 512 to 5 linear layer, which is the only part that is trained. This is fast and data-efficient, but the backbone cannot adapt its features to the fine texture of leaf lesions.

	Custom CNN	ResNet-18 (frozen)
Initialisation	Random (from scratch)	ImageNet pretrained
Trainable parameters	about 51.5 million	2,565 (final layer only)
Where most parameters live	Flatten-to-512 fully connected layer	Frozen convolutional backbone
Regularisation	BatchNorm, Dropout 0.25	BatchNorm layers (frozen statistics)
Saved checkpoint	Best validation epoch (7)	Best validation epoch (10)

Design note
The custom network is small in depth but not small in parameters: most of its roughly 51.5 million weights sit in the single fully connected layer that follows flattening. It is therefore a compact feature extractor with a large classifier head, not a parameter-light model.

METHOD

4. Training Setup

Setting	Value
Framework	PyTorch and Torchvision
Hardware	NVIDIA T4 GPU (Google Colab)
Epochs	10 per model
Optimiser	Adam, learning rate 0.001 (ResNet-18: final layer parameters only)
Loss	Cross-entropy
Batch size	32
Model selection	Checkpoint saved whenever validation accuracy improves
Time per epoch	about 13.7 s (Custom CNN), about 11.2 s (ResNet-18)
Reproducibility	Random seed 42 for the split

Each epoch runs a full training pass followed by a validation pass. The validation set is used only to choose the best checkpoint, and the test set is touched exactly once at the end, so the reported test accuracy is an unbiased estimate for these data.

RESULTS

5. Results and Evaluation

Both models learn quickly. The custom CNN starts lower on the first epoch but overtakes ResNet-18 within two epochs and reaches its best validation accuracy of 97.33% at epoch 7. ResNet-18 improves more slowly and steadily, peaking at 96.27% at epoch 10.

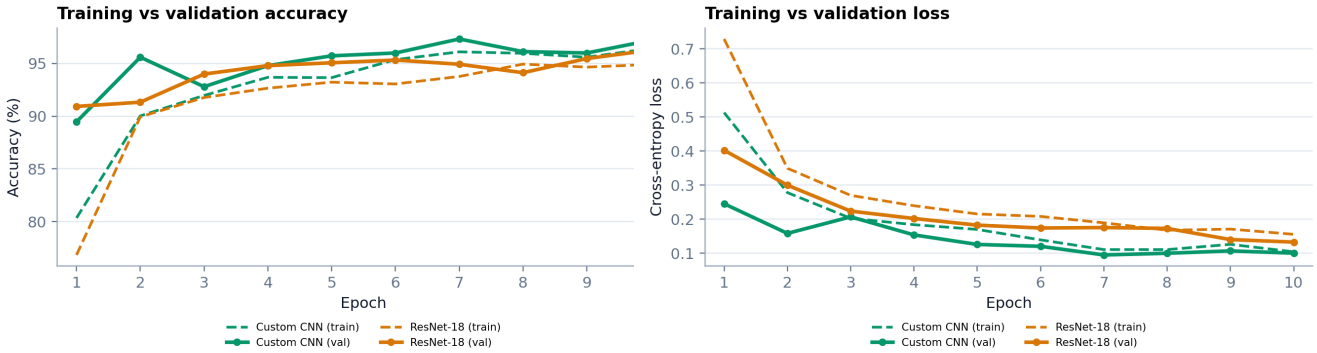
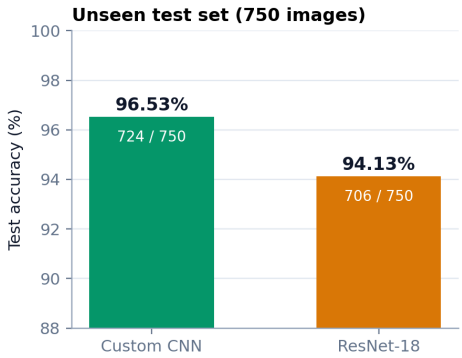


Figure 3. Accuracy (left) and loss (right) over ten epochs. Solid lines are validation, dashed lines are training.

Epoch-by-epoch results

Epoch	Custom train	Custom val	ResNet train	ResNet val
1	80.34%	89.47%	76.83%	90.93%
2	90.03%	95.60%	89.94%	91.33%
4	93.69%	94.80%	92.66%	94.80%
5	93.66%	95.73%	93.23%	95.07%
7	96.11%	**97.33%** (best)	93.77%	94.93%
10	96.43%	97.20%	94.91%	**96.27%** (best)



Final test evaluation

On the 750 unseen test images the custom CNN classified **724** correctly (**96.53%**) and ResNet-18 classified **706** correctly (**94.13%**). That is 26 errors versus 44, a reduction of about 41% in mistakes for the custom model.

Accuracy is reported on one random split. Per-class precision, recall, and a confusion matrix were not computed in the notebook, so class-level behaviour is discussed qualitatively below.

INSIGHTS

6. Analysis and Discussion

Why the custom CNN wins here

ImageNet features describe everyday objects. Leaf disease depends on subtle local patterns such as lesion edges, ring structure, and small colour shifts. Because every layer of the custom CNN is trained, its filters specialise in exactly those patterns. The frozen ResNet-18 can only re-weight its generic features through a single linear layer, which explains its slower progress and lower ceiling. This is a plausible explanation rather than a proven one; an ablation that fine-tunes the last ResNet block would test it directly.

Generalisation

Validation and training curves stay close for both models, and validation accuracy sometimes exceeds training accuracy. This is expected because training images are augmented and dropout is active while training, whereas validation is clean. There is no sign of strong overfitting within ten epochs.

Trade-offs between the engines

Aspect	Custom CNN	ResNet-18
Test accuracy	96.53%	94.13%
Training speed	Slightly slower per epoch	Faster (one layer updated)
Trainable weights	about 51.5 M	2,565
Needs pretrained weights	No	Yes (ImageNet)
Adaptation to leaf texture	Full	Limited by frozen backbone

Takeaway
Offering both engines in the app is more than a demo feature. When the two models agree, confidence in the diagnosis rises; when they disagree, the app tells the user to inspect the leaf manually.

DEPLOYMENT

7. Application and System Design

The trained checkpoints are served by a modular Streamlit application. The interface is split so that models, configuration, and presentation can each change without touching the others.

File	Responsibility
app.py	Page layout, sidebar controls, tabs, and the scan workflow.
config.py	Class names, paths, confidence threshold, labels, and the disease knowledge base.
model.py	Model definitions, checkpoint loading, and cached prediction.
ui.py	HTML components: result cards, probability bars, and report generation.
styles.css	All visual styling.
best_custom_cnn.pth / best_resnet18.pth	Trained model weights.

Key features

- **Three inference modes:** Custom CNN, ResNet-18, or Dual Benchmark, selected from the sidebar.
- **Safety threshold:** predictions below 65% confidence are shown as *Unrecognized* instead of being guessed.
- **Dual agreement banner:** reports whether both engines agree, disagree, or are unsure.
- **Probability bars and gauge:** the full distribution over the five classes for every scan.
- **Disease library and action plans:** signs, severity, and recommended care for each class.
- **Scan history and reports:** a session log, plus a downloadable text report for confident results.
- **Robustness:** cached predictions, automatic reload when checkpoints change, and friendly messages for unreadable images or missing weights.

Responsible use
The app provides AI-assisted guidance only. Critical treatment decisions should be confirmed with a qualified agronomist.

OUTLOOK

8. Limitations and Future Work

Limitations

- **Dataset realism:** PlantVillage photographs are taken on plain backgrounds in controlled conditions. Accuracy on field photos with shadows, soil, and overlapping leaves is likely to be lower.
- **Synthetic rejection class:** *Not Leaf* contains simple generated patterns, so real-world non-leaf photos may still receive a confident leaf label.
- **Narrow scope:** three diseases and one crop. Other tomato diseases (for example leaf mould, septoria, and viruses) are not represented.
- **Single split and metrics:** one random split and overall accuracy only; no cross-validation, confusion matrix, or per-class metrics.
- **Comparison fairness:** ResNet-18 had a frozen backbone, so the result compares a fully trained custom model with a feature-extraction baseline, not with a fine-tuned ResNet.

Future work

Direction	Description
Fine-tuned ResNet	Unfreeze later blocks with a lower learning rate for a fairer comparison.
Richer evaluation	Add a confusion matrix, per-class precision and recall, and k-fold cross-validation.
Real-world data	Add field images and realistic negatives (soil, hands, other plants) to the Not Leaf class.
Multi-crop expansion	Extend to potato, pepper, corn, apple, and grape.
Edge deployment	Quantise or export to ONNX or a mobile-friendly backbone for offline phone use.
Outbreak mapping	Tag scans with location to map regional disease spread.

SUMMARY

9. Conclusion

FloraScan AI demonstrates a complete deep learning workflow: curating and balancing a dataset, designing an out-of-distribution class, training and comparing two architectures, and shipping the result as a usable application. The custom three-block CNN achieved **96.53%** test accuracy against **94.13%** for a frozen ResNet-18, and the confidence threshold and dual-engine check make the system more cautious about its own answers. The main next steps are testing on real field imagery and a more detailed per-class evaluation.

96.53%	94.13%	724 / 750	65%
Custom CNN	ResNet-18	Correct (Custom)	Safety threshold

REFERENCES

References

[1] Hughes, D. P. and Salathe, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. arXiv:1511.08060.

[2] He, K., Zhang, X., Ren, S. and Sun, J. (2016). Deep residual learning for image recognition. CVPR.

[3] Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. ICLR.

[4] Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. ICML.

[5] Paszke, A. et al. (2019). PyTorch: An imperative style, high-performance deep learning library. NeurIPS.

[6] Food and Agriculture Organization of the United Nations. Plant production and protection: pests and diseases.

Asad Ali Sajjad
SU92-BSCSM-F23-167 | BSCS-7A, 7th Semester
Deep Learning, Dr. Munir Ahmad